

MODULE 7 –Spatial Filter: Low Pass Filter, High Pass Filter, Point Detection, Line Detection, Edge Detection

A. PURPOSE

- Students are able to understand the concept of Spatial Filters
- Students are able to distinguish types of filter Spatial
- Students are able to create a simple image spatial filter using 2D convolution .

B. TOOLS AND MATERIALS

1. PC/LAPTOP
2. Github
3. Google Colaborator

C. THEORY REVIEW

C.1. Filter Process

In general , the process of filtering an image is a mechanism that can change optical , electronic or digital signals according to certain criteria. Filtering is a way to extract certain part of a set of data, by eliminating the parts data is not desirable. The aim of the performed process filter on an image is making the image become more visible, or seem more obvious to be analyze. In digital image processing, filters are used to:

1. Hold high frequency on imagea, smoothening images(smoothing)
2. Hold low frequency on images, clarifiesor detects edges in the image.

Filter in image processing can be done in the frequency domain and spatial domain .

- a. **Frequency Domain.** Filtering the image in the frequency scopes will change the image into frequency domain, multiplying by a function of the frequency filter, transform back the results to spatially space scope. The function of the filter works to reduce (attenuates) the frequency specific and amplify the frequency of the other , or can also be interpreted to pass (receive) component frequency specific and eliminate (reject) the other frequency. For instance lowpass filter function means to pass components of low frequencies. Low Pass Filter generates value 1 for a frequency less than the threshold and 0 to the other so that the obtained image blurred (soft/smooth).
- b. **Spatial Domain.** Spatial domain filtering is the process of manipulating a collection of pixels from an image to produce a new image. Filtering domain spatial is one of the methods that are used in many fields for a variety of applications, especially for the improvement of the quality of the image and fix image. Filtering the image of the space scope of spatial are doing convolution of image input $f(i,j)$ with the function of the filter $h(i,j)$, where the function of the filter that used to be simulated in the form of kernel discrete certain.

$$g(i,j) = h(i,j) \odot f(i,j)$$

C.2. Convolution Process

Convolution is the sum of the multiplication of each point in the kernel by each point in the input function. Kernel operate as a shift in the function of the input image input $f(x)$. Total multiplication of each point on the function that is the result of convolution that is expressed by $g(x)$. The kernel uses the concept of neighboring pixels, where the kernel matrix is made with the assumption that the value of a pixel can be influenced by their neighbor pixels. Neighbor pixel is the number of pixels which is adjacent to the direct (adjacent) with a pixel center. Illustration of neighboring pixels can be seen in Figure 1.

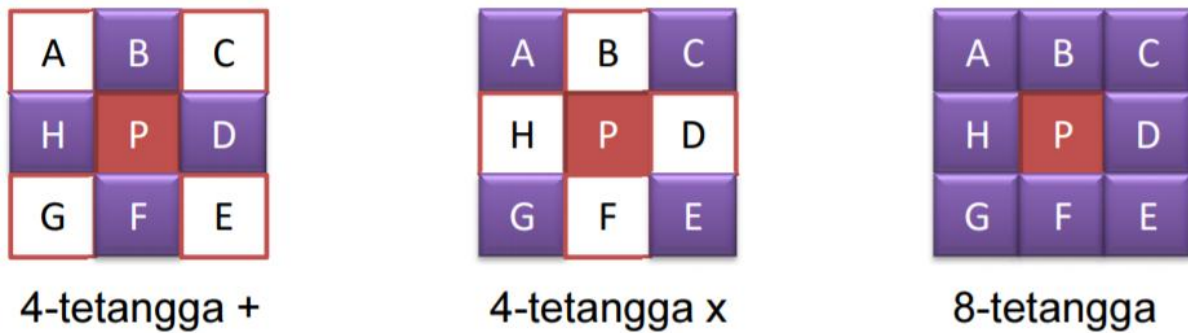


Figure 1. Illustration of neighboring pixels of pixel P

In digital image processing, convolution is carried out in two dimensions in an image, as shown by the equation:

$$g(x, y) = f(x, y) \odot h(x, y) = \sum_{a=-\infty}^{\infty} \sum_{b=-\infty}^{\infty} f(a, b) g(x - a, y - b)$$

The illustration of the convolution process from the above equation is shown in Figure 2.

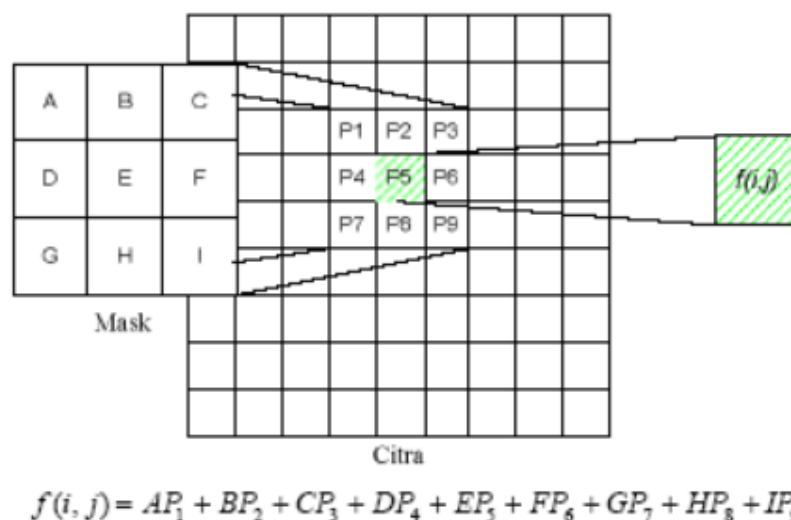


Figure 2. Illustration of the convolutional process

For example, there is an input image matrix and a kernel matrix as follows :

5	5	6	6
5	4	4	7
0	0	2	2
0	1	1	3

(a) Citra input

$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$
$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$
$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$

(b) Kernel

So the results of the calculation of convolution area are :

$$g(2,2) = 5 \times \frac{1}{4} + 5 \times \frac{1}{4} + 6 \times \frac{1}{4} + 5 \times \frac{1}{4} + 4 \times \frac{1}{4} + 4 \times \frac{1}{4} + 0 \times \frac{1}{4} + 0 \times \frac{1}{4} + 2 \times \frac{1}{4}$$

$$= 1,25 + 1,25 + 1,5 + 1,25 + 1 + 1 + 0 + 0 + 0,5 = 7,75$$

Convolution is carried out until all the pixels of the input image are subjected to the convolution calculation . Here is an example of the stages of the process of convolution in a matrix image of size 5x5 with a kernel size of 3x3.

$$f(x,y) = \begin{bmatrix} 4 & 4 & 3 & 5 & 4 \\ 6 & 6 & 5 & 5 & 2 \\ 5 & 6 & 6 & 6 & 2 \\ 6 & 7 & 5 & 5 & 3 \\ 3 & 5 & 2 & 4 & 4 \end{bmatrix} \quad g(x,y) = \begin{bmatrix} 0 & -1 & 0 \\ -1 & *4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

note : mark * indicates the position (0.0) of the kernel

The convolution operation $f(x,y) \odot h(x,y)$ are as follows:

1. Place the kernel in the upper left corner , then calculate the pixel value at the position (0,0) of the kernel.

4	4	3	5	4
6	6	5	5	2
5	6	6	6	2
6	7	5	5	3
3	5	2	4	4

→

	3			

Hasil konvolusi = 3. Nilai ini dihitung dengan cara berikut:

$$(0 \times 4) + (-1 \times 4) + (0 \times 3) + (-1 \times 6) + (4 \times 6) + (-1 \times 5) + (0 \times 5) + (-1 \times 6) + (0 \times 6) = 3$$

2. Move the kernel one pixel to the right , then calculate the pixel value at the position (0,0) of the kernel

4	4	3	5	4
6	6	5	5	2
5	6	6	6	2
6	7	5	5	3
3	5	2	4	4

→

	3	0		

Hasil konvolusi = 0. Nilai ini dihitung dengan dengan cara berikut:

$$(0 \times 4) + (-1 \times 3) + (0 \times 5) + (-1 \times 6) + (4 \times 5) + (-1 \times 5) + (0 \times 6) + (-1 \times 6) + (0 \times 6) = 0$$

3. Move the kernel one pixel to the right , then calculate the pixel value at the position (0,0) of the kernel

4	4	3	5	4
6	6	5	5	2
5	6	6	6	2
6	7	5	5	3
3	5	2	4	4

→

	3	0	2	

Hasil konvolusi = 2. Nilai ini dihitung dengan cara berikut:

$$(0 \times 3) + (-1 \times 5) + (0 \times 4) + (-1 \times 5) + (4 \times 5) + (-1 \times 2) + (0 \times 6) + (-1 \times 6) + (0 \times 2) = 2$$

4. Furthermore shift kernel one pixel to the bottom, then began to perform the convolution of the left image . Every convolution process, slide kernel one pixel to the right and calculate the value of the pixel at position (0,0) of the kernel .

(i)

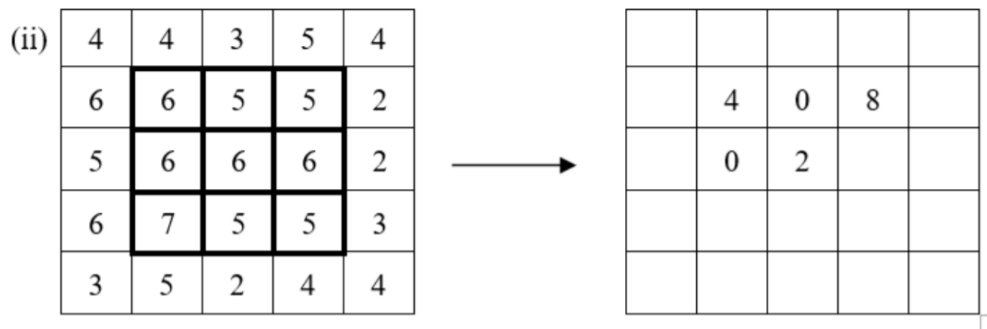
4	4	3	5	4
6	6	5	5	2
5	6	6	6	2
6	7	5	5	3
3	5	2	4	4

→

	3	0	2	
	0			

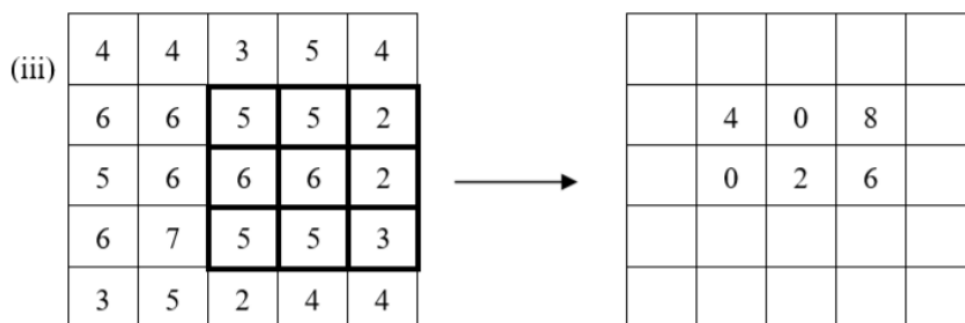
Hasil konvolusi = 0. Nilai ini dihitung dengan cara berikut:

$$(0 \times 6) + (-1 \times 6) + (0 \times 5) + (-1 \times 5) + (4 \times 6) + (-1 \times 6) + (0 \times 6) + (-1 \times 7) + (0 \times 5) = 0$$



Hasil konvolusi = 2. Nilai ini dihitung dengan cara berikut:

$$(0 \times 6) + (-1 \times 5) + (0 \times 5) + (-1 \times 6) + (4 \times 6) + (-1 \times 6) + (0 \times 7) + (-1 \times 5) + (0 \times 5) = 2$$



Hasil konvolusi = 6. Nilai ini dihitung dengan cara berikut:

$$(0 \times 5) + (-1 \times 5) + (0 \times 2) + (-1 \times 6) + (4 \times 6) + (-1 \times 2) + (0 \times 5) + (-1 \times 5) + (0 \times 3) = 6$$

5. By the same way as in the above, then the pixels in row three convolve to produce :

	4	0	8	
	0	2	6	
	6	0	2	

Problems arise when the pixel convolve is in the edge of the image (border), because some of the convolution coefficients is not able to be positioned on these image pixels (the "hang" effect) as shown in the Figure 3. "hang" problems like this always happens in the left, right, top, and bottom of image pixels edges.

4	4	3	5	4	?
6	6	5	5	2	?
5	6	6	6	2	?
6	7	5	5	3	
3	5	2	4	4	

Figure 3. An example of the hang effect on a convolution process

The solution to a problem early this are :

1. Edge pixels are ignored, do not convolve. This solution is widely used in the usages of image processing functions. Using this alternative, all the edge of the pixels value remains the same as the image of origin , such as the example in Figure 4.

4	4	3	5	4
6	4	0	8	2
5	0	2	6	2
6	6	0	2	3
3	5	2	4	4

Gambar 4. Edge pixels (un-shaded) are not convolve

2. Duplicate element of the image. For example, elements of the first column copied into column $M + 1$, also on the contrary , the convolution can be performed on the edge as illustrated in Figure 5.

5	5	5			
5	5	5	6	6	
5	5	4	4	7	
	0	0	2	2	
	0	1	1	3	

Figure 5. Illustration of the pixel value duplication process

3. Elements that are marked with a "?" In Figure 3 , it is assumed that the value is 0 or some other constant , so that the edge pixel convolution can be carried out , as illustrated in Figure 6.

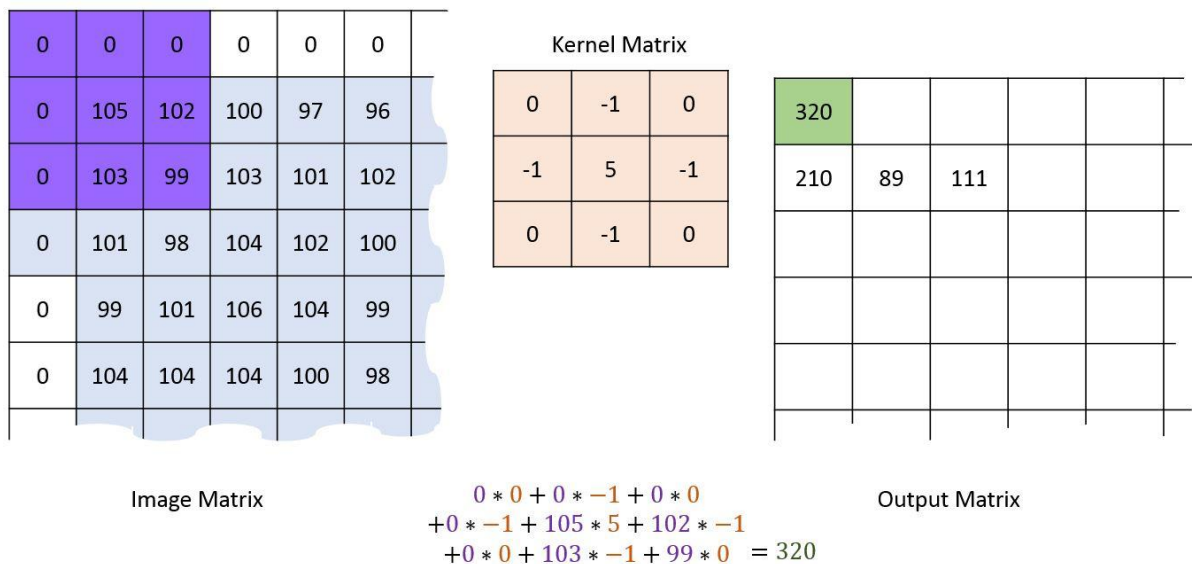


Figure 6. Illustration of giving the value 0 to the input image

Solutions with three approaches in the above assumes part of the edge of the image width is very small (only one pixel) relative compared denagn size of the image , so the pixels on the edge does not show the effect from our naked eyes.

The following is a convolutional process algorithm

```

For x = 0 to image_width-1
  For y = 0 image_height-1
    z(x,y) = 0
    For k1 = 0 to filter_width-1
      For k2 = 0 to filter_height-1
        z(x,y) = z(x,y) + H(k1,k2)*Img_pixel(x+k1, y+k2)
      next k2
    next k1
  next y
next x
    
```

The pseudocode for the convolution process is as follows :

```
void konvolusi(citra Image, citra ImageResult, imatriks Mask, int N, int M)
/* Mengkonvolusi citra Image yang berukuran N x M dengan mask 3 x 3. Hasil
konvolusi disimpan di dalam matriks ImageResult.
*/
{ int i, j;

  for (i=1; i<=N-3; i++)
    for(j=1; j<=M-3; j++)
      ImageResult[i][j]=
        Image[i-1][j-1]*Mask[0][0] +
        Image[i-1][j+1]*Mask[0][1] +
        Image[i-1][j]*Mask[0][2] +
        Image[i][j-1]*Mask[1][0] +
        Image[i][j]*Mask[1][1] +
        Image[i][j+1]*Mask[1][2] +
        Image[i+1][j-1]*Mask[2][0] +
        Image[i+1][j]*Mask[2][1] +
        Image[i+1][j+1]*Mask[2][2];
}
```

The convoluted pixels are elements (i, j). Eight pieces of pixels adjacent to the pixel (i, j) is as follows :

$i-1, j-1$	$i-1, j$	$i-1, j+1$
$i, j-1$	i, j	$i, j+1$
$i+1, j-1$	$i+1, j$	$i+1, j+1$

C.3. Spatial Filter Type

The results of the filter spatially determined by the element matrix of the kernel, and can produce effects that vary depending on the image Input. Some of the types of spatial filters are :

- Sharpening (sharpening)
- Blurring (blurring)
- Emboss
- Edge detection (detection edge)

C.3.1. Sharpening

Kernel for sharpening / sharpening the image using the principle that the intensity of the pixels centers should be strengthened in the direction that is opposite to its neighbors . Example of a kernel for image enhancement :

0	-1	0
-1	5	-1
0	-1	0

courtesy

<http://docs.gimp.org/en/plugin-in-convmatrix.html>

1	1	1
1	-8	1
1	1	1

courtesy

<http://www.imagesincontext.com/IICTFeatures/convolution-filter.htm>

While examples of sharpening results can be seen in Figure 7.

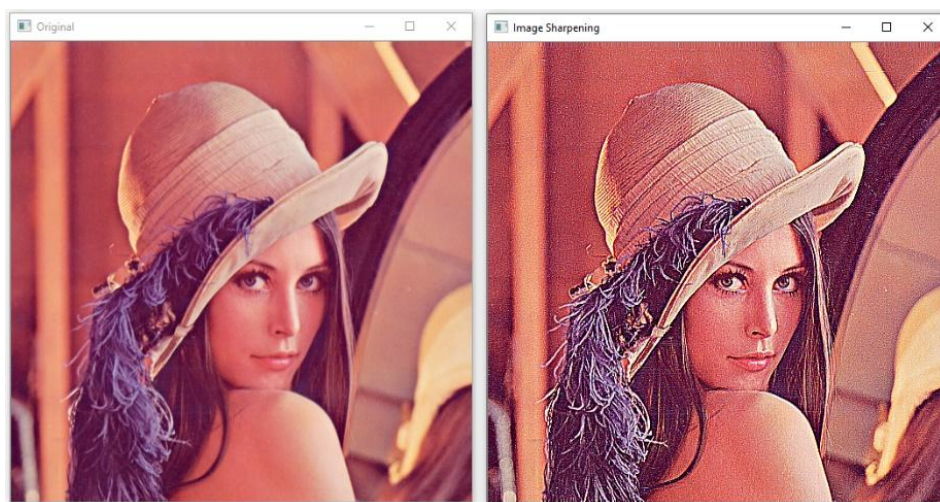


Figure 7. Example of a Sharpening Filter

C.3.2. Blurring

The kernel for blurring / blurring uses the principle that the central pixel value should be made closer to neighboring pixels (reducing discrepancies). An example of a kernel for blurring images, one can use Gaussian Blurring:

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \quad \text{or} \quad \frac{1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$$

While examples of blurring results can be seen in Figure 9 .

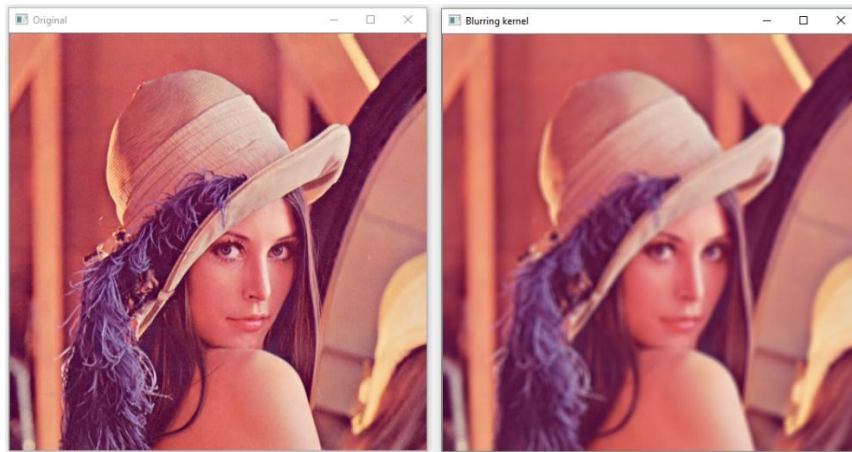


Figure 9 . Example of a Gaussian Blurring Filter

C.3.3. Emboss

The kernel for embossing is based on the principle of reinforcing edges in one particular direction , without losing any other color. The direction of strengthening is indicated by the change from negative to positive elements . Example kernel to emboss:

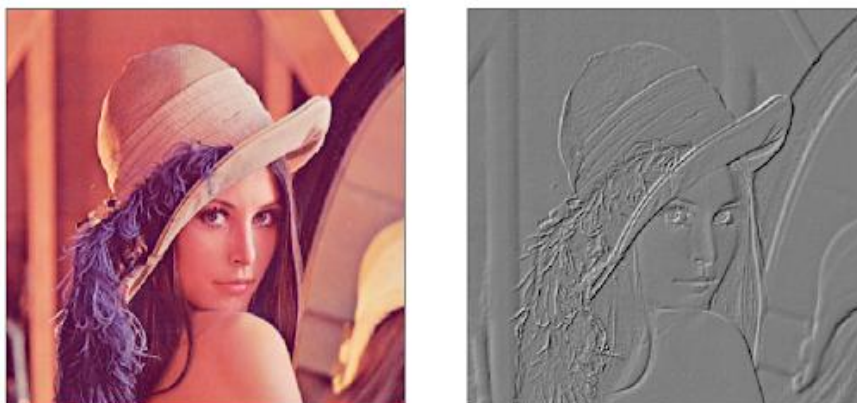
-2	-1	0
-1	1	1
0	1	2

courtesy
<http://docs.gimp.org/en/plugin-convmatrix.html>

1	0	0
0	0	0
0	0	-1

courtesy
<http://www.imagesincontext.com/IICFeatures/convolution-filter.htm>

While an example of the embossed result can be seen in Figure 10.



Here 's a few examples of other kernel filter base:

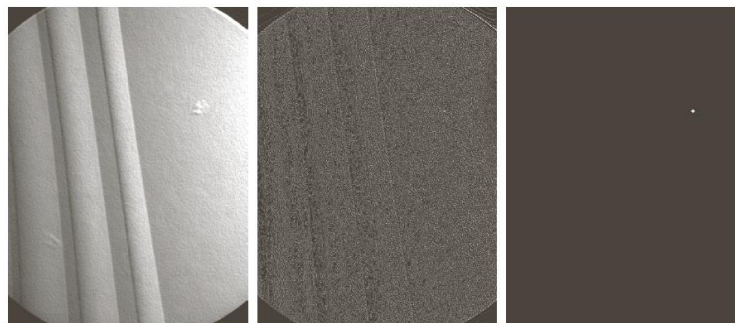
- *Average Filter 3x3* = $\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$

- *Low Pass Filter* = $\frac{1}{12} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 4 & 1 \\ 1 & 1 & 1 \end{bmatrix}$

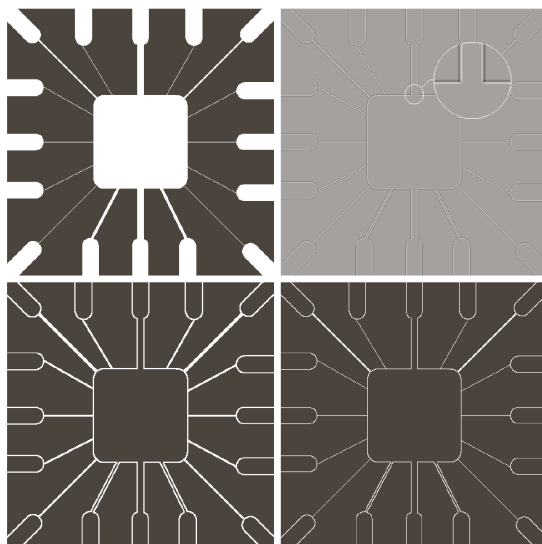
- *High Pass Filter* = $\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 3 \\ -3 & 0 & 1 \end{bmatrix}$

- **Point Detection**

1	1	1
1	-8	1
1	1	1



- **Line Detection**



-1	-1	-1	2	-1	-1
2	2	2	-1	2	-1
-1	-1	-1	-1	-1	2

Horizontal

+45°

-1	2	-1	-1	-1	2
-1	2	-1	-1	2	-1
-1	2	-1	2	-1	-1

Vertical

-45°

- **Edge Detection**

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

-1	0	0	-1
0	1	1	0

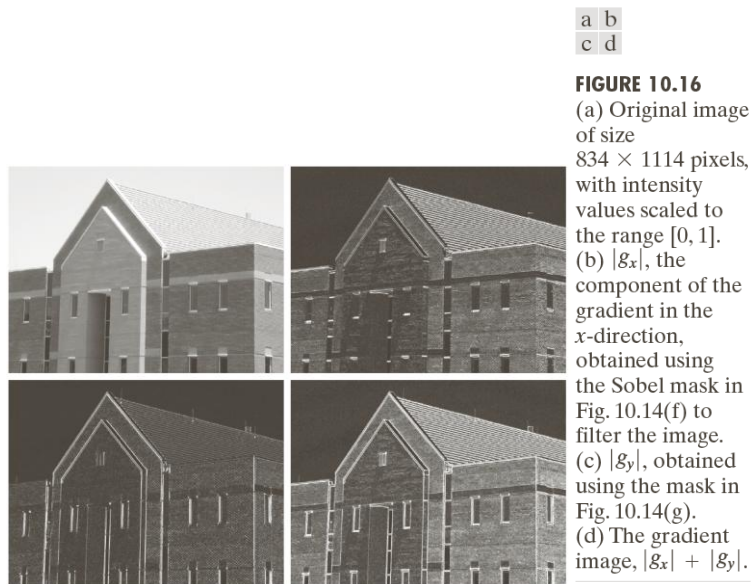
Roberts

-1	-1	-1	-1	0	1
0	0	0	-1	0	1
1	1	1	-1	0	1

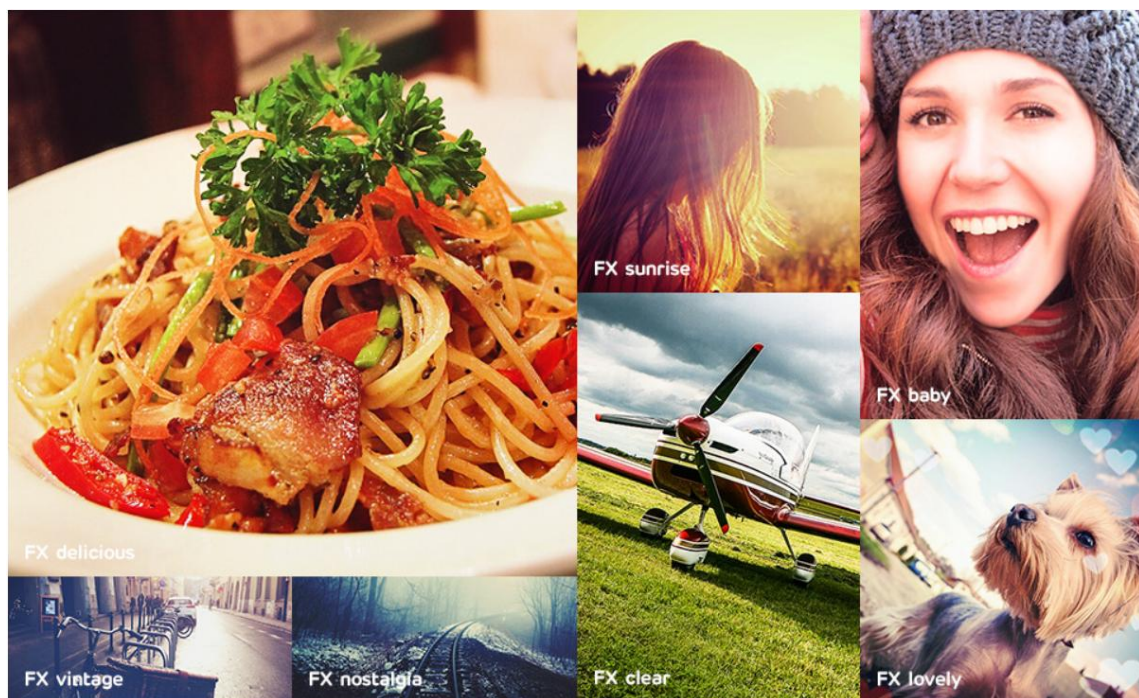
Prewitt

-1	-2	-1	-1	0	1
0	0	0	-2	0	2
1	2	1	-1	0	1

Sobel

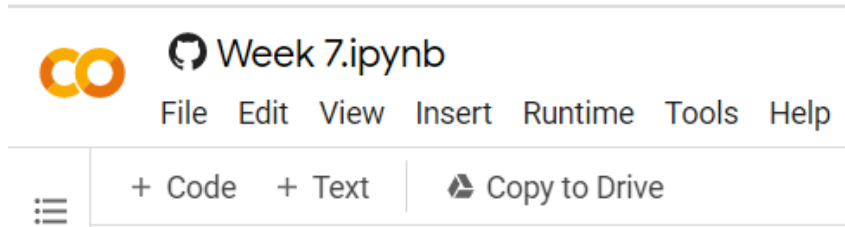


When this almost all application processing image have filter specifically each of which is very attractive to users . Figure below is an example of a filter that is owned by mobile Line Camera Application.



D. PRAKTIKUM FILTER

1. Make convolution function using algorithms that have been described in Part C, without using a library or method of convolution of OpenCV.
2. The following are steps that can be taken :
 - a. Create a notebook just on google colab, and give the name Week7.ipynb. Save a copy on the github account as in the previous module .



- b. Access the files contained on the drive and import some of the libraries that are needed

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[ ] import numpy as np
import matplotlib.pyplot as plt
import cv2 as cv
import math
from google.colab.patches import cv2_imshow
from PIL import Image as im
```

- c. Create a convolutional function . Note : the parameters used may be modified . For example , only uses parameters and kernel image only , or image, kernel, and padding.

▼ Konvolusi tanpa Library

Membuat fungsi konvolusi

Fungsi konvolusi yang dibuat memiliki parameter berupa:

1. citra masukan,
2. kernel berupa matriks untuk memfilter citra,
3. nilai stride / besarnya pergeseran untuk setiap konvolusi,
4. nilai pad yang akan ditambahkan pada citra

```
[ ] def convolution2d(image, kernel, stride, padding):
```

- d. Load the image to be processed and turn it into a gray image

Load Image yang akan diproses

```
[ ] img = cv.imread('/content/drive/MyDrive/Polinema/Kuliah/PCVK/Images/mandrill.tiff')
img_gray = cv.cvtColor(img,cv.COLOR_BGR2GRAY)
```

- e. Determine the kernel that will be used , for example, the kernel to filter sharpening as follows :

Menentukan kernel yang akan digunakan

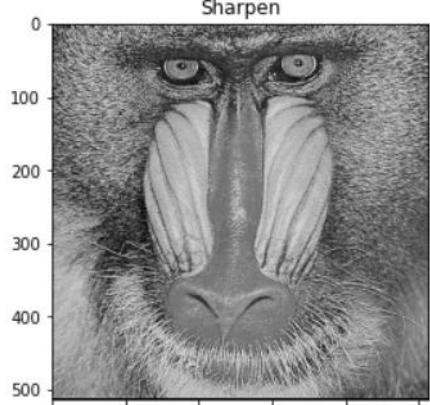
```
#image sharpen
kernel_sharpen = np.array([[0,-1,0],
                           [-1,5,-1],
                           [0,-1,0]])
```

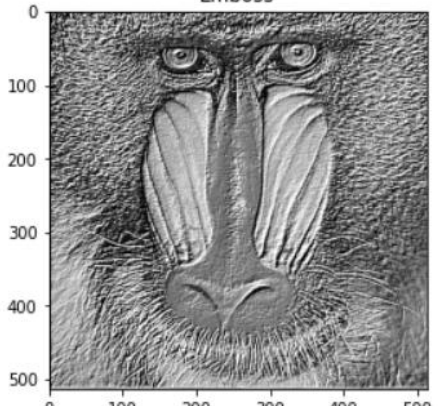
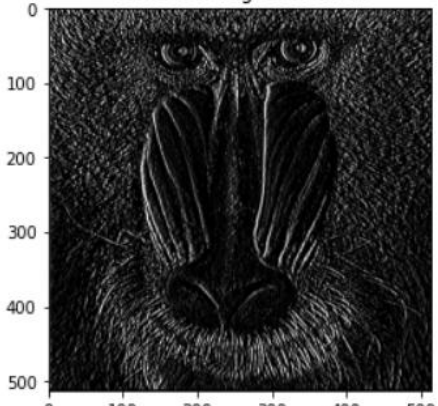
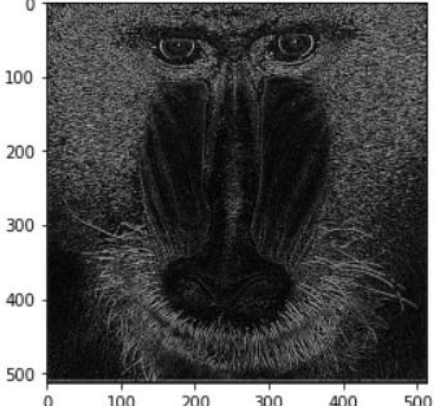
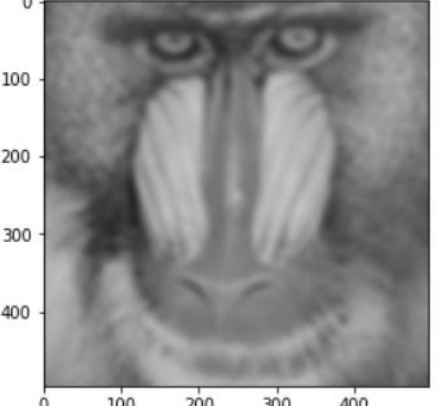
- f. Calling a function convolution that has been created previously , and displays the results konvolusinya :

Memanggil fungsi konvolusi dan menerapkan setiap filter yang telah ditentukan

```
convolution2d(img_gray,kernel_sharpen,1,2)
```

3. Create Image Filter for Average filter, low pass filter, high pass filter, and some of the following filters :

Operasi	Kernel	Hasil
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	Sharpen 

Emboss	$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$	Emboss 
Left Sobel Edge Detection	$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$	Left Sobel Edge Detection 
Canny Edge Detection	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	Canny Edge Detection 
21x21 Gaussian Blur	To generate a Gaussian kernel, you can use the following code : <pre>sigma=math.sqrt(kernel_size) gaussian_kernel = cv.getGaussianKernel(kernel_size , sigma) gauss_kernel = gaussian_kernel @ gaussian_kernel.transpose()</pre>	21x21 Gaussian Blur 

E. FILTER LIBRARY AND MODERN FILTER

In the following experiments we will look at several filters using libraries, modern filters used in CNNs, modern filters with a combination of several traditional filters (without Deep Learning).

Experiment 1:

In this first experiment, we'll create Gaussian, Sharpen, and Canny filters using the OpenCV filter2d library. We'll apply these filters to an RGB image. At the beginning of the code, there's a `show_side_by_side` function used to display images side by side.

```
# Fungsi tampil berdampingan
def show_side_by_side(images, titles, figsize=(15,5)):
    plt.figure(figsize=figsize)
    for i, (img, title) in enumerate(zip(images, titles)):
        if len(img.shape) == 2: # grayscale
            plt.subplot(1, len(images), i+1)
            plt.imshow(img, cmap="gray")
        else: # color
            plt.subplot(1, len(images), i+1)
            plt.imshow(cv.cvtColor(img, cv.COLOR_BGR2RGB))
        plt.title(title)
        plt.axis("off")
    plt.show()

img = cv.imread("/content/drive/MyDrive/Kuliah/PCVK Genap 2023/Images/lena.jpg")
img_gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)

blur = cv.GaussianBlur(img, (7,7), 1)
edges = cv.Canny(cv.cvtColor(img, cv.COLOR_BGR2GRAY), 100, 200)
sharpen_kernel = np.array([[0,-1,0],[-1,5,-1],[0,-1,0]])
sharpened = cv.filter2D(img, -1, sharpen_kernel)
show_side_by_side([img, blur, sharpened, edges],
                  ["Asli", "Gaussian Blur", "Sharpened", "Canny Edge Detection"])
```

Here are the results displayed:



Experiment 2:

In the following experiment 2, we will implement modern filtering from the OpenCV Library. The two filters used are Bilateral Filtering and Guided Filtering. **Bilateral filtering** is a non-linear filtering method widely used to smooth images while maintaining sharp edges. Unlike conventional Gaussian blur, which only considers the spatial distance between pixels, bilateral filtering also considers differences in color intensity. This way, pixels that are close and have similar colors to the center of the window will receive greater weight in the calculation, while

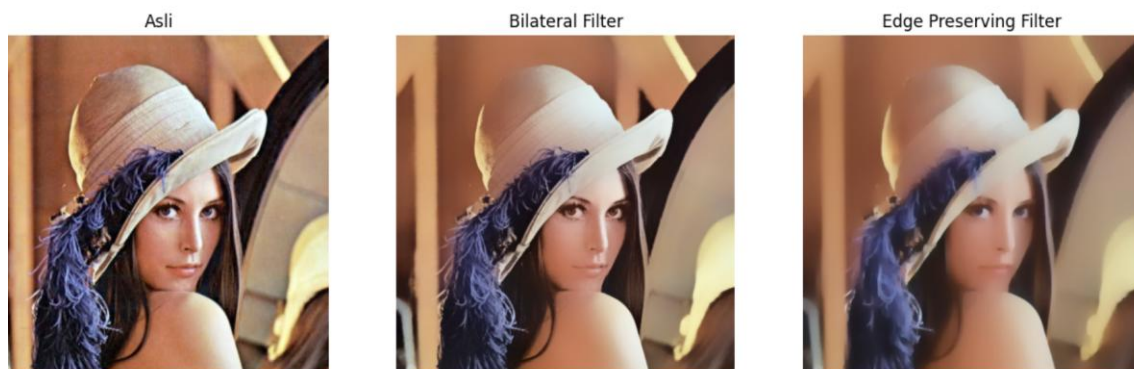
pixels with contrasting colors (for example, at the edges) will have their influence reduced. As a result, the image becomes smoother in flat areas, while boundaries and important details are maintained. Although it produces good quality, bilateral filtering is relatively slow due to the complex weight calculation. In practice, this filter is widely used for purposes such as photo quality improvement, noise reduction, and as the basis for *beauty filters* to soften skin texture without blurring facial contours.

Guided filtering is a more modern and efficient technique. This filter is based on the assumption that, within a local window, the filtering result can be represented as a linear function of the guide image. This means that each output pixel is calculated by considering the linear relationship between the pixel values in the input image and the pixel values in the guide image. If the input image and the guide image are the same, the guided filter will act similarly to a bilateral filter but with much faster calculations and smoother results. Another advantage, the guided filter can use different images as guides, thus being able to direct the filtering process according to needs. Due to this property, guided filters are often used in various advanced applications such as *HDR tone mapping*, detail enhancement, *image matting*, *feathering*, and *depth map* refinement in stereo vision systems. In general, guided filters offer a balance between good edge preservation, smooth visual quality, and computational efficiency, and are therefore considered an improvement on the bilateral filtering approach.

```
#Filter Modern dari OpenCV
# Bilateral Filter (edge-preserving)
bilateral = cv.bilateralFilter(img, 50, 100, 100)

# Edge Preserving Filter (alternatif Guided Filter)
edge_preserve = cv.edgePreservingFilter(img, flags=1, sigma_s=100, sigma_r=0.9)

show_side_by_side([img, bilateral, edge_preserve],
                  ["Asli", "Bilateral Filter", "Edge Preserving Filter"])
```



Experiment 3:

This experiment will examine the filtering process in a CNN (the Feature Map section). Run the code several times and observe the output. What conclusions can you draw from these results?

```
#Filter Feature Map yang digunakan pada CNN, Lakukan running code bagian ini beberapa kali dan perhatikan hasilnya
import torch
import torch.nn as nn

class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 4, kernel_size=3, stride=1, padding=1)

    def forward(self, x):
        return self.conv1(x)

model = SimpleCNN()

# Ubah gambar ke tensor
img_tensor = torch.tensor(img_gray, dtype=torch.float32).unsqueeze(0).unsqueeze(0) / 255.0

# Hasil CNN
with torch.no_grad():
    features = model(img_tensor)

# Visualisasi feature maps
feature_maps = [features[0,i].numpy() for i in range(features.shape[1])]
show_side_by_side([img_gray] + feature_maps, ["Asli (Gray)" + [f"Feature {i+1}" for i in range(len(feature_maps))]])
```



Experiment 4:

This experiment will implement the Beauty and Vintage effects commonly used in popular apps. The filters used are a combination of traditional filters. It's important to note that popular app filters may not use the same method. Popular apps may use the GenAI model with training data to provide more accurate results.

```
# =====
# 1. Beauty Filter
# =====
# Step 1: Smoothing kulit dengan bilateral filter
smooth = cv.bilateralFilter(img, d=15, sigmaColor=75, sigmaSpace=75)

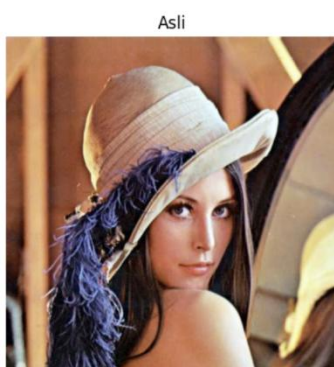
# Step 2: Unsharp masking (pertajam mata/bibir)
gaussian = cv.GaussianBlur(smooth, (0,0), 3)
sharpened = cv.addWeighted(smooth, 1.5, gaussian, -0.5, 0)

# Step 3: Brightness & contrast
alpha = 1.2 # contrast
beta = 15 # brightness
beauty = cv.convertScaleAbs(sharpened, alpha=alpha, beta=beta)

# =====
# 2. Old/Vintage Filter
# =====
# Step 1: Sepia tone
sepia_kernel = np.array([[0.272, 0.534, 0.131],
                        [0.349, 0.686, 0.168],
                        [0.393, 0.769, 0.189]])
sepia = cv.transform(img, sepia_kernel)
sepia = np.clip(sepia, 0, 255).astype(np.uint8)

# Step 2: Vignette
rows, cols = img.shape[:2]
kernel_x = cv.getGaussianKernel(cols, cols*0.6)
kernel_y = cv.getGaussianKernel(rows, rows*0.6)
kernel = kernel_y * kernel_x.T
mask = kernel / kernel.max()
vignette = np.copy(sepia)
for i in range(3):
    vignette[:, :, i] = vignette[:, :, i] * mask

# Step 3: Noise/Grain
noise = np.random.normal(0, 15, vignette.shape).astype(np.int16)
old_img = np.clip(vignette.astype(np.int16) + noise, 0, 255).astype(np.uint8)
```



Experiment 5:

Experiment 5 will show you an anime/cartoon filter using a combination of traditional filters.


```
#Filter Anime / Cartoon
# Step 1: Edge detection (pakai median blur dulu agar gambar menjadi Lebih halus)
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
gray_blur = cv.medianBlur(gray, 7)
edges = cv.adaptiveThreshold(gray_blur, 255,
                             cv.ADAPTIVE_THRESH_MEAN_C,
                             cv.THRESH_BINARY, 9, 9)

# Step 2: Bilateral filter untuk smoothing warna
color = cv.bilateralFilter(img, d=9, sigmaColor=200, sigmaSpace=200)

# Step 3: Gabungkan (cartoonize)
cartoon = cv.bitwise_and(color, color, mask=edges)

# Tampilkan
show_side_by_side([img, cartoon], ["Asli", "Cartoon Filter"])
```

Asli



Cartoon Filter



Experiment 6:

In Experiment 6 an example of the Night Filter will be shown.

```
#Night Filter
img = cv.imread("/content/drive/MyDrive/Kuliah/PCVK Genap 2023/Images/djawatan.jpg")
img_gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)

# Step 1: Gelapkan (contrast turun, brightness negatif)
night = cv.convertScaleAbs(img, alpha=0.6, beta=-40)

# Step 2: Tambah bias biru
blue_tint = np.full_like(night, (50, 0, 100)) # BGR
night = cv.addWeighted(night, 0.8, blue_tint, 0.2, 0)

# Step 3: Efek glow di area terang dengan filter2D (blur kernel)
kernel = np.ones((15,15), np.float32) / 225
glow = cv.filter2D(night, -1, kernel)

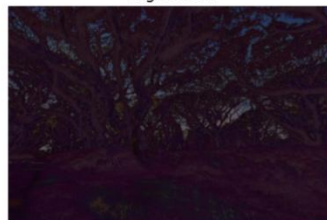
# Kombinasikan asli + glow
night_glow = cv.addWeighted(night, 0.7, glow, 0.3, 0)

show_side_by_side([img, night, night_glow],
                  ["Asli", "Night Filter", "Night + Glow (filter2D)"])
```

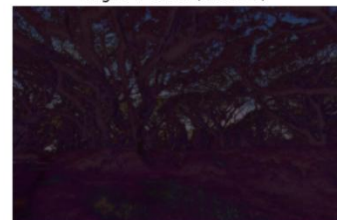
Asli



Night Filter



Night + Glow (filter2D)



Experiment 7

Experiment 7 shows the Morning and Morning Filters added with a fog effect.

```
#Filter Suasana pagi dan Kabut
# =====
# Step 1: Kurangi kontras & cerahkan
# =====
alpha = 0.9 # contrast
beta = 20 # brightness
soft = cv.convertScaleAbs(img, alpha=alpha, beta=beta)

# =====
# Step 2: Tambahkan warm tone (kemerahan / oranye)
# =====
warm_tint = np.full_like(soft, (40, 70, 120)) # BGR
pagi = cv.addWeighted(soft, 0.8, warm_tint, 0.2, 0)

# =====
# Step 3: Tambahkan haze (kabut tipis) dengan filter2D
# =====
# Kernel blur Gaussian-like untuk menciptakan efek kabut
kernel = cv.getGaussianKernel(3, 3)
kernel = kernel @ kernel.T # jadikan 2D kernel
kabut = cv.filter2D(pagi, -1, kernel)

# tambah layer putih untuk kabut lebih nyata
white_layer = np.full_like(pagi, 255)
kabut = cv.addWeighted(kabut, 0.7, white_layer, 0.3, 0)

show_side_by_side([img, pagi, kabut],
                  ["Asli", "Pagi", "Pagi + Kabut"])
```

